PHISHGUARD

EMAIL THREAT ASSESSMENT

Programming Language: C | Data Structure: Circular Queue

SUBMITTED BY

Sr. No.	Student Name	Roll No.
1	Sunkara Venkata Sai Jagadeesh	39
2	Sudhanshu Pendalwar	46
3	Sameer Pandey	38

B.Tech (AI & Data Science), Second Year

Queueing incoming emails and scoring each one for phishing risk, first come, first served

1. Aim

To develop an Email Threat Assessment System, **PhishGuard**, that uses a circular queue to hold incoming emails and assesses each one in arrival order, giving it a phishing risk score and a clear verdict of Safe, Suspicious or Phishing.

2. Abstract

PhishGuard is a menu-driven application that manages incoming emails described by a sender address, a subject and a message body. New emails join the rear of a fixed-size circular queue, and the analyst assesses the email at the front, so that mail is handled strictly in the order it arrived (FIFO). Each assessed email receives a threat score between 0 and 100, calculated from suspicious keywords, insecure links and a risky or malformed sender address. The project handles queue overflow and underflow and keeps a session summary of how many emails were found safe, suspicious or phishing.

3. Objectives and Features

- Receive a new email and add it to the queue (enqueue).
- Assess the next email in arrival order (dequeue).
- Peek at the next email without removing it.
- Display all pending emails from front to rear.
- Assess every pending email in one go.
- Score each email and classify it as Safe, Suspicious or Phishing.
- Detect queue overflow (full inbox) and underflow (empty inbox).
- Show a session summary of all verdicts.

4. How the Circular Queue Works

A queue follows the First In, First Out rule: the email that arrives first is assessed first. A plain array queue wastes space because slots freed at the front are never reused. A **circular queue** fixes this by wrapping the indexes around the end of the array.

```
rear  = (rear + 1) MOD MAX      /* enqueue: move rear forward   */
front = (front + 1) MOD MAX     /* dequeue: move front forward  */
```

The variable **count** tracks how many emails are waiting. The queue is empty when count is 0 and full when count equals MAX (10). Using count avoids the confusion between the full and empty states that happens when only front and rear are compared.

Threat scoring. Each email is scored by the rules below and the score is capped at 100.

Check	Points	Reason
Each suspicious keyword found in subject or body (urgent, verify, password, suspended, click, winner, lottery, otp, bank, login, prize, confirm)	+10 each	Typical pressure and credential-theft words
Link starts with http:// (not https://)	+15	Insecure link
Sender address ends with .xyz, .top or .tk	+20	Domains often used in scams
Sender address has no @ symbol	+20	Malformed sender

Score	Verdict
0 to 29	SAFE
30 to 59	SUSPICIOUS
60 to 100	PHISHING

5. Algorithm

1. Start the program and initialize the queue (front = 0, rear = -1, count = 0).
2. Display the menu of available operations.
3. For receiving an email, check that the queue is not full, accept the sender, subject and body, then enqueue it at the rear.
4. For assessing the next email, check that the queue is not empty, dequeue the email at the front, calculate its threat score and print the verdict.
5. For peeking, show the front email without removing it.
6. For display, traverse from front for count elements, wrapping with MOD MAX, and print each email.
7. For assessing all emails, repeat the dequeue and assessment steps until the queue is empty.
8. For the summary, print the number of safe, suspicious and phishing emails and the pending count.
9. Stop the program when the user selects Exit.

6. C Program

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX 10        /* capacity of the circular queue */
#define KEYWORDS 12   /* number of suspicious keywords  */

struct Email {
    int  id;
    char sender[50];
    char subject[100];
    char body[200];
};

/* ---------- Circular queue (the data structure used) ---------- */
struct Email queue[MAX];
int front = 0, rear = -1, count = 0;

int nextId = 1;
int safeCount = 0, suspiciousCount = 0, phishingCount = 0;

const char *keyword[KEYWORDS] = {
    "urgent", "verify", "password", "suspended", "click", "winner",
    "lottery", "otp", "bank", "login", "prize", "confirm"
};

int isEmpty() { return count == 0; }
int isFull()  { return count == MAX; }

/* ENQUEUE: add an email at the rear of the queue */
void enqueue(struct Email e) {
    rear = (rear + 1) % MAX;       /* wrap around */
    queue[rear] = e;
    count++;
}

/* DEQUEUE: remove and return the email at the front */
struct Email dequeue() {
    struct Email e = queue[front];
    front = (front + 1) % MAX;     /* wrap around */
    count--;
    return e;
}

/* Read a full line (spaces allowed) */
void readLine(char s[], int size) {
    if (fgets(s, size, stdin) == NULL) { s[0] = '\0'; return; }
    s[strcspn(s, "\n")] = '\0';
}

void toLowerStr(char dest[], const char src[]) {
    int i;
    for (i = 0; src[i] != '\0'; i++)
        dest[i] = tolower((unsigned char)src[i]);
    dest[i] = '\0';
}

int endsWith(const char s[], const char tail[]) {
    int ls = strlen(s), lt = strlen(tail);
    return ls >= lt && strcmp(s + ls - lt, tail) == 0;
}

/* Threat score (0-100). *hits = number of suspicious keywords found */
int assess(struct Email *e, int *hits) {
    char text[320], sender[50];
    char *at;
    int score = 0, i;

    *hits = 0;
    snprintf(text, sizeof(text), "%s %s", e->subject, e->body);
```

```
    toLowerStr(text, text);
    toLowerStr(sender, e->sender);

    for (i = 0; i < KEYWORDS; i++) {         /* +10 per keyword */
        if (strstr(text, keyword[i])) {
            score += 10;
            (*hits)++;
        }
    }
    if (strstr(text, "http://"))             /* insecure link */
        score += 15;

    at = strchr(sender, '@');
    if (at == NULL)                          /* malformed sender */
        score += 20;
    else if (endsWith(at, ".xyz") || endsWith(at, ".top") || endsWith(at, ".tk"))
        score += 20;                         /* risky domain */

    return score > 100 ? 100 : score;
}

const char *verdict(int score) {
    if (score >= 60) return "PHISHING";
    if (score >= 30) return "SUSPICIOUS";
    return "SAFE";
}

void receiveEmail() {
    struct Email e;

    if (isFull()) {
        printf("Inbox queue is full (overflow). Assess some emails first.\n");
        return;
    }
    printf("Enter sender address: ");
    readLine(e.sender, sizeof(e.sender));
    printf("Enter subject: ");
    readLine(e.subject, sizeof(e.subject));
    printf("Enter message body: ");
    readLine(e.body, sizeof(e.body));

    if (strlen(e.sender) == 0) {
        printf("Sender cannot be empty.\n");
        return;
    }
    e.id = nextId++;
    enqueue(e);
    printf("Email #%d received and queued (position %d).\n", e.id, count);
}

void report(struct Email e) {
    int hits, score = assess(&e, &hits);
    const char *v = verdict(score);

    printf("\n----- THREAT REPORT : Email #%d -----\n", e.id);
    printf("From    : %s\n", e.sender);
    printf("Subject : %s\n", e.subject);
    printf("Keywords: %d suspicious\n", hits);
    printf("Score   : %d / 100\n", score);
    printf("Verdict : %s\n", v);

    if (score >= 60)      phishingCount++;
    else if (score >= 30) suspiciousCount++;
    else                  safeCount++;
}

/* Assess the email at the FRONT of the queue (first come, first served) */
void assessNext() {
    if (isEmpty()) {
        printf("No emails waiting (queue is empty).\n");
        return;
    }
    report(dequeue());
}
void assessAll() {
```

```
    if (isEmpty()) {
        printf("No emails waiting (queue is empty).\n");
        return;
    }
    while (!isEmpty())
        report(dequeue());
}

void peekNext() {
    if (isEmpty()) {
        printf("No emails waiting (queue is empty).\n");
        return;
    }
    printf("Next to assess -> #%d from %s | %s\n",
           queue[front].id, queue[front].sender, queue[front].subject);
}

void displayQueue() {
    int i;
    printf("\n===== PENDING EMAILS (%d/%d) =====\n", count, MAX);
    if (isEmpty()) {
        printf("Queue is empty.\n");
        return;
    }
    for (i = 0; i < count; i++) {
        struct Email *e = &queue[(front + i) % MAX];
        printf("%d. #%-3d %-25s %s\n", i + 1, e->id, e->sender, e->subject);
    }
}

void summary() {
    printf("\n===== SESSION SUMMARY =====\n");
    printf("Safe       : %d\n", safeCount);
    printf("Suspicious : %d\n", suspiciousCount);
    printf("Phishing   : %d\n", phishingCount);
    printf("Pending    : %d\n", count);
}

int main() {
    char line[10];
    int choice;

    do {
        printf("\n===== PHISHGUARD =====\n");
        printf("1. Receive Email (Enqueue)\n");
        printf("2. Assess Next Email (Dequeue)\n");
        printf("3. Peek Next Email\n");
        printf("4. Display Pending Emails\n");
        printf("5. Assess All Pending Emails\n");
        printf("6. Session Summary\n");
        printf("0. Exit\n");
        printf("Enter your choice: ");
        readLine(line, sizeof(line));
        choice = atoi(line);

        switch (choice) {
            case 1: receiveEmail(); break;
            case 2: assessNext();   break;
            case 3: peekNext();     break;
            case 4: displayQueue(); break;
            case 5: assessAll();    break;
            case 6: summary();      break;
            case 0: printf("Exiting PhishGuard.\n"); break;
            default: printf("Invalid choice.\n");
        }
    } while (choice != 0);
    return 0;
}
```

7. Sample Output

```
===== PHISHGUARD =====
1. Receive Email (Enqueue)
2. Assess Next Email (Dequeue)
3. Peek Next Email
4. Display Pending Emails
5. Assess All Pending Emails
6. Session Summary
0. Exit
Enter your choice: 1
Enter sender address: support@secure-bank.xyz
Enter subject: URGENT: Verify your account
Enter message body: Your account is suspended. Click http://secure-bank.xyz/login to confirm your password.
Email #1 received and queued (position 1).

Enter your choice: 1
Enter sender address: prof.mehta@college.edu
Enter subject: Mini project submission
Enter message body: Please submit your report by Friday.
Email #2 received and queued (position 2).

Enter your choice: 4

===== PENDING EMAILS (2/10) =====
1. #1   support@secure-bank.xyz   URGENT: Verify your account
2. #2   prof.mehta@college.edu    Mini project submission

Enter your choice: 2

----- THREAT REPORT : Email #1 -----
From    : support@secure-bank.xyz
Subject : URGENT: Verify your account
Keywords: 8 suspicious
Score   : 100 / 100
Verdict : PHISHING

Enter your choice: 2

----- THREAT REPORT : Email #2 -----
From    : prof.mehta@college.edu
Subject : Mini project submission
Keywords: 0 suspicious
Score   : 0 / 100
Verdict : SAFE

Enter your choice: 2
No emails waiting (queue is empty).
```

8. Performance Analysis

Operation	Average Time	Worst Case
Receive email (enqueue)	O(1)	O(1)
Assess next email (dequeue + scoring)	O(L)	O(L)
Peek next email	O(1)	O(1)
Display pending emails	O(n)	O(n)
Assess all pending emails	O(n × L)	O(n × L)

Here n is the number of queued emails and L is the length of the email text, because scoring scans the subject and body for a fixed set of keywords. Enqueue, dequeue and peek only change an index and copy one record, so they take constant time. Because the queue is circular, no elements are shifted when an email is removed, and freed slots are reused. The queue holds at most 10 emails, so it uses a small fixed amount of memory.

9. Advantages

- Emails are assessed fairly in the order they arrive (FIFO).
- Enqueue and dequeue take constant time.
- The circular design reuses freed space with no shifting of data.
- Overflow and underflow are detected and reported clearly.
- Simple, explainable scoring rules that are easy to extend.
- Demonstrates a practical use of the queue data structure.

10. Future Scope

- Load the keyword list from a file so it can be updated without recompiling.
- Use a priority queue so high-risk emails are assessed first.
- Save the threat reports to a file so records persist.
- Detect look-alike domains such as paypa1.com.
- Import emails from a text or CSV file.
- Build a graphical user interface.

11. Conclusion

PhishGuard demonstrates the practical use of a circular queue for processing incoming emails in arrival order. Wrapping the front and rear indexes with MOD keeps every operation at constant time and reuses space efficiently, while a simple scoring model turns each email into a clear Safe, Suspicious or Phishing verdict. The project shows how a basic data structure can support a real security task.